COMP4135 SEM Final Group Report

Group 2 (Tactical Cards)

Team members

Guangjun HU (20511787)
Hongshuo HU (20808603)
Cheng YE (20513323)
Zehan CHEN (20807750)

School of Computer Science
University of Nottingham

Submission Date: TBC - confirm on Moodle

Abstract

Tactical Cards is a Unity-based tactical card combat prototype designed for teenagers and casual strategy players who benefit from readable, small-scope decision making. The delivered project provides a complete single-battle loop with a title screen, a playable grid encounter, a result screen, deterministic enemy response, and a themed runtime HUD. The design focuses on one compact tactical scenario rather than a larger campaign structure so that interaction order, board state, card effects, and player feedback remain easy to understand. This report concentrates on the system itself: the requirements that shape the battle loop, the architecture used to implement it, the runtime interaction model, the user interface design, the current validation results, and the main technical limitations and extension paths.

Contents

Abstract **1**

1 Introduction **5**
1.1 Motivation 5
1.2 Aims and Objectives 5
1.3 Delivered Contribution 6

2 Background and Related Work **7**
2.1 Design Context 7
2.2 Audience Fit 7
2.3 Reference Strategy 8

3 Requirements Engineering **9**
3.1 Requirements Elicitation 9
3.2 Personas and Scenarios 10
3.3 Requirement Evolution 10
3.4 Final Requirements Specification 13
3.5 Requirements Validation 14

4 Development **15**
4.1 System Design 15
4.2 Implementation Strategy and Iteration Process 17
4.3 Prototype and UI Structure 19
4.4 Reserved Updated Screenshot Set 20
4.5 Code Structure 23
4.6 Unity Engine-Level Implementation Details 25
4.7 Simulation Design 25
4.8 UI Design 26

5 Test & System Evaluation **28**
5.1 Test Strategy 28
5.2 Current Rerun State 29
5.3 Issue Discovery and Closure Loop 30
5.4 Code Coverage 32
5.5 Evaluation and Technical Self-Critique 33

6 Extensibility & Maintenance **35**
6.1 Delivered Maintainability Measures 35
6.2 Deferred Work and Future Evolution 35

7 Problem & Reflections **37**
7.1 Technical Problem 37
7.2 Project Management 37
7.3 Team Management 38

8 Conclusion **40**

Reference **41**

A User Manual **42**
A.1 Purpose 42
A.2 Opening the Project 42
A.3 Starting a Battle 42
A.4 Core Interaction Flow 42
A.5 Reading the Battle HUD 43
A.5.1 Top Bar 43
A.5.2 Left Panel 43
A.5.3 Bottom Hand Panel 43
A.5.4 World-space Unit Feedback 43
A.6 Card Reference 43
A.7 Understanding the Turn Loop 44
A.8 Ending the Battle 44
A.9 Troubleshooting 44

B Meeting Minutes **45**
B.1 Evidence Status 45
B.2 Recovered Early Implementation Checkpoint 45
B.2.1 Purpose 46
B.2.2 Recovered Decisions 46
B.2.3 Recovered Follow-up 46
B.3 Kickoff Meeting 46
B.3.1 Context 46
B.3.2 Decisions 46
B.4 Scope Alignment Meeting 46
B.4.1 Context 47
B.4.2 Decisions 47
B.5 Validation Planning Meeting 47
B.5.1 Attendance 47
B.5.2 Agenda 47
B.5.3 Key Decisions 47
B.5.4 Action Items 48

C Test Case **49**
C.1 Validation Evidence Register 49
C.2 Coverage Snapshot 49
C.3 Representative Automated Test Inventory 50
C.4 Manual Smoke Checklist 51

D Development Planning **52**
D.1 Iterative Delivery Model 52
D.2 Backlog Reprioritisation Logic 53

E Version Control **55**
E.1 Workflow Summary 55
E.2 Repository Graph Snapshot 55
E.3 Workflow Limits 56

F Docs **57**
F.1 Supporting Document Register 57

Chapter 1

Introduction

Tactical Cards is a Unity-based tactical card battle prototype designed for teenagers and casual strategy players. The delivered software provides a complete single-battle session: the player enters through a start screen, controls allied units on a grid, plays tactical cards, observes enemy response, and reaches a clear result state.

1.1 Motivation

The project began from one simple design goal: each turn should ask the player to make a small number of visible and meaningful tactical decisions without requiring a large content set or a complicated ruleset. Instead of aiming for a campaign-heavy tactics game, the design focuses on one encounter in which positioning, range, timing, and action order matter.

The tactical card format was chosen because it combines two readable systems. First, a grid-based battle makes spatial decisions visible on screen. Second, a small hand of action cards introduces variety without turning the project into a deckbuilding or collection-heavy game. This makes the game strategically meaningful while still remaining approachable for the intended audience.

1.2 Aims and Objectives

The project has four main objectives.

1. Deliver a complete playable battle loop in Unity, including player turn, enemy turn, and battle-end handling.
2. Keep the gameplay narrow enough that the core interactions remain readable, stable, and testable.
3. Build the system so that turn flow, card resolution, grid interaction, and runtime UI are understandable at code level.
4. Improve interface clarity and feedback so that a first-time player can understand what to do without outside explanation.

Success is therefore defined less by feature quantity than by the completeness of one polished vertical slice. A successful outcome means that a player

can launch the game, start a battle, understand the interaction order, observe enemy response, finish the encounter, and then replay or return cleanly to the menu.

1.3 Delivered Contribution

The delivered build is intentionally modest in content, but complete in structure. It contains:

- a full start-to-result session flow;
- a 6x6 tactical battle with card-driven actions and deterministic enemy response;
- a panel-based HUD with direct action feedback, prompts, and world-space health bars;
- automated validation for deterministic runtime behaviour, plus manual smoke for presentation-heavy UI;
- a code structure that separates flow, battle rules, UI, and presentation cleanly.

The project did not remain static from concept to completion. Early design assumptions were revised as playable builds exposed onboarding, readability, and session-flow problems. The report therefore does not present the final system as if it had emerged fully formed. Chapter 3 explains how the requirement set evolved from the earliest design baseline into the delivered implementation, and Chapter 4 explains how that evolution was handled through iterative development rather than a once-through sequence.

Chapter 2

Background and Related Work

This chapter situates the game within a small-scale tactical design space and explains why that design direction is suitable for the intended audience. The focus remains practical: the aim is to justify the core interaction model rather than to provide a broad survey of commercial tactics games.

2.1 Design Context

The delivered game sits between a small-scale tactical skirmish game and a card-driven command system. It is not intended to compete with content-heavy strategy RPGs or large collectible card games. Instead, it adopts the parts of those genres that support clear turn structure, visible positional play, and short decision cycles.

The core design context is a lightweight tactical grid battle in which the player controls a very small team and spends a limited hand of cards each turn. Positioning, range, targeting, and action ordering are the main decision variables, while content breadth is intentionally constrained. The project therefore uses cards as a readable action language, not as a collectible system with large pools, rarity, or synergy chains.

Unity was selected because the game depends on both world-space interaction and screen-space feedback. The board, tile highlights, unit placement, and combat feedback all live in the same runtime environment as the HUD, menu flow, and result presentation. That makes Unity a better fit than a document-only prototype or a purely web-first interaction model.

2.2 Audience Fit

The target group is teenage and casual strategy players. First, the game is structured around short repeated decisions rather than long uninterrupted sessions. This fits the idea that games can support competence and agency when the player is given clear choices and immediate consequences [6]. Each turn in Tactical Cards asks the player to choose one unit, one card, and one legal target, which keeps the decision loop visible instead of overwhelming.

Second, the strategic layer remains compact but genuine. Adachi and

Willoughby report a positive longitudinal relationship between strategic videogame play and self-reported problem-solving skills among adolescents [1]. Tactical Cards does not claim to be an educational intervention in the strict research sense, but it does adopt the same design intuition: ask the player to reason about position, range, and action order under bounded resources.

Third, the project intentionally avoids features that would weaken its fit for the target audience in a compact prototype. There is no large narrative gate, no deckbuilding complexity, no rarity economy, and no large campaign state to memorise. Instead, the player sees a small board, a limited hand, explicit prompt text, and direct feedback after each action. Granic et al. note that the benefits of games are often tied to how clearly challenge, mastery, and feedback are structured [4]; the final product therefore prioritises readable battle flow over content breadth.

2.3 Reference Strategy

The project uses external references selectively and at two different levels. At the gameplay level, it borrows only broad genre patterns: grid occupancy, ranged versus melee positioning, and card-labelled actions. No external gameplay framework is imported into the runtime. The battle loop, flow coordination, grid logic, enemy response, and HUD behaviour are authored within the project codebase.

At the visual level, the interface uses open-licensed panel and icon resources, while the board dressing, layout logic, and gameplay-facing presentation behaviour remain locally implemented. The visual surface is therefore themed, but the runtime interaction design is still original to this project.

Chapter 3

Requirements Engineering

This chapter defines the final system requirements in terms of user needs, interaction scenarios, and the delivered functional and non-functional targets of the game. It also explains how the requirement set evolved from the earliest concept into the completed build, because the differences between the original design assumptions and the final implementation are important rather than accidental.

3.1 Requirements Elicitation

Three stakeholder perspectives shaped the requirement set.

Stakeholder	**Core concern**	**Design implication**
Teenage end user	The game should be easy to start, clear to read, and still strategically meaningful.	The final build keeps the rules, board size, hand size, and feedback surfaces deliberately compact.
Developer / maintainer	The system should be implementable, testable, and extensible without destabilising the core loop.	The final project remains one battle slice with clear script ownership and separate system layers.
Designer	Positioning, range, timing, and action order should remain visible and tunable.	The final build uses a small card set and deterministic enemy behaviour to keep balancing readable.

Table 3.1: Stakeholder concerns reflected in the delivered design.

These perspectives established the earliest design baseline: one compact tactical session for teenage players, readable enough for a first-time user, and small enough to be implemented and validated properly. That baseline remained stable, but several requirements changed in form as the team moved from concept diagrams to a playable system.

3.2 Personas and Scenarios

Design item	Core concern	Design implication
Persona 1 First-time teenage player	Learn the battle flow quickly, understand unit/card/target selection, and receive clear feedback after each action.	Drove the prompt stack, visible energy label, disabled-card reasons, result flow, and world-space health bars.
Persona 2 Developer / maintainer	Keep scope manageable, preserve clarity, and align implementation with system responsibilities.	Drove the single-battle vertical slice, modular script layout, and explicit manager boundaries.
Scenario 1 First battle interaction	The player selects a friendly unit, selects a card, and then selects a valid target while the system limits invalid interaction.	Remains the core interaction path for both implementation and HUD feedback work.
Scenario 2 Understanding turn results	The player ends the turn and needs to understand what the enemy just did and why.	Remains the main reason for keeping the battle log, enemy-phase summary, and visible health feedback.

Table 3.2: Personas and scenarios that shape the current system.

3.3 Requirement Evolution

The finished game differs in several important ways from the earliest design assumption, but the changes are coherent rather than contradictory. The original concept emphasised tactical combat on a small board. The delivered build still does that, but the implementation path showed that a battle-only prototype was not enough for a complete submission or for a readable first-time experience. As a result, several requirements evolved from minimal placeholders into explicit user-facing obligations.

Area	Initial design assumption	Delivered form	Reason for change	Validation evidence
Session flow	The earliest playable target focused on entering battle directly.	The final build includes title, battle, and result states.	The battle slice needed clearer entry, completion, replay, and menu return behaviour.	Flow tests and PlayMode smoke
HUD feedback	Status text and log output were considered sufficient for the prototype.	The final HUD exposes prompts, energy, disabled-card reasons, and world-space health bars.	Playable smoke showed that hidden state and log-only feedback slowed understanding.	‘BattleUITests‘, ‘CardBut-ton-ViewTests‘, smoke
Card variety	The first card set only needed the minimum actions to prove the turn loop.	The final build adds ‘Dash‘, ‘Long Strike‘, and ‘Recover‘ to the base actions.	The battle needed slightly deeper tactical choice without broadening into deckbuilding.	Card and deck tests, manual battle smoke
Presentation	Primitive units and minimal UI were acceptable while the core loop was still unstable.	The final build uses themed UI, dressed boards, and silhouette-based unit presentation.	A submission-ready slice needed stronger readability and visual cohesion.	Visual smoke and runtime checks
Validation	Early validation focused mainly on proving core battle correctness.	The final package combines EditMode tests, PlayMode smoke, and issue-driven UX smoke.	Later UI and flow work introduced player-visible defects that required more than logic-only tests.	Execution summary, smoke records, issue closure

Table 3.3: How the delivered system differs from the earliest design assumptions.

The final specifications should be regarded as a refined version of the original intent. The core objective of the project has always been compact tactical card battles targeted at teenage users. As issues emerged during the development process and new ideas came to light, the requirements were iterated accordingly.

3.4 Final Requirements Specification

ID	Requirement	Delivered meaning in the final build	Acceptance check
FR-01	The game shall provide a complete front-end flow.	The player can move from title screen to battle and then to a result state with replay or return-to-menu actions.	Visible title, battle, and result states verified in flow tests and smoke.
FR-02	The game shall initialise one playable tactical battle on a 6x6 grid.	The scene creates a 6x6 board, places player and enemy units, and starts a valid battle loop.	Bootstrap, grid, and PlayMode readiness tests.
FR-03	The player turn shall refresh resources and draw a hand of cards.	Turn start resets the action budget and refreshes the runtime hand.	‘TurnManagerTests‘, ‘DeckManagerTests‘.
FR-04	The player shall select a unit, choose a card, and act on a valid target only.	The HUD and board flow enforce valid ordering and surface invalid-target feedback.	‘BattleUITests‘, ‘CardResolverTests‘, smoke.
FR-05	Command cards shall support movement, attack, defence, repositioning, and recovery.	The final card set includes Move, Dash, Strike, Long Strike, Guard, Push, and Recover.	Card and deck suites plus manual battle smoke.
FR-06	Enemy units shall execute visible and deterministic turns.	Enemy AI targets the nearest valid player and either attacks or advances accordingly.	‘EnemyAITests‘, turn-loop tests, smoke.
FR-07	The game shall expose clear victory and defeat states.	Battle outcome transitions into a result panel with replay and menu actions.	Flow tests and result-state smoke.
FR-08	The runtime HUD shall expose key battle state clearly.	The final HUD shows prompts, hand, energy, battle log, card availability, and world-space health bars.	‘BattleUITests‘, ‘CardButtonViewTests‘, HUD smoke.
NFR-01	Critical combat information shall remain directly visible during battle.	Turn state, remaining energy, selected-unit context, card availability, and unit HP are visible without opening extra menus.	HUD smoke and layout regression tests.
NFR-02	The demo shall run through one full session without requiring debug-only	A user can go from title to result through the normal flow path.	PlayMode smoke and manual smoke.

3.5 Requirements Validation

The requirements are validated through a combination of direct automated tests, PlayMode smoke checks, and manual visual smoke. This layered approach is necessary because the project contains both deterministic game rules and presentation-heavy UI concerns.

Requirement group	**Validation approach**	**Primary evidence**
Battle bootstrap and flow	EditMode bootstrap tests plus PlayMode smoke	‘GameRootTests‘, ‘GameFlowControllerTests‘, ‘BattleFlowPlayModeTests‘
Cards and turn loop	EditMode rule-level tests	‘TurnManagerTests‘, ‘DeckManagerTests‘, ‘CardResolverTests‘
Grid and unit state	EditMode placement, movement, damage, and occupancy tests	‘GridManagerTests‘, ‘UnitControllerTests‘, ‘TileViewTests‘
Enemy response	EditMode AI tests plus turn-loop integration checks	‘EnemyAITests‘, ‘TurnManagerTests‘
HUD and interaction feedback	EditMode UI tests plus manual smoke for final layout	‘BattleUITests‘, ‘CardButtonViewTests‘, HUD smoke
Usability refinements	Scenario-based walkthrough followed by smoke retest	Runtime verification and layout regression checks

Table 3.5: How the final requirement groups are validated.

The resulting specification is not only a feature list. It records what the player must be able to do, how clearly the game must communicate state, and how the project moved from an early concept into a more complete and better-validated tactical slice.

Chapter 4

Development

This chapter describes the architecture, implementation structure, runtime interaction model, simulation rules, UI design, and the iterative process used to deliver the final build.

4.1 System Design

The runtime architecture is split by responsibility rather than by scene object alone. This keeps the battle systems readable and makes it easier to test core behaviour directly.

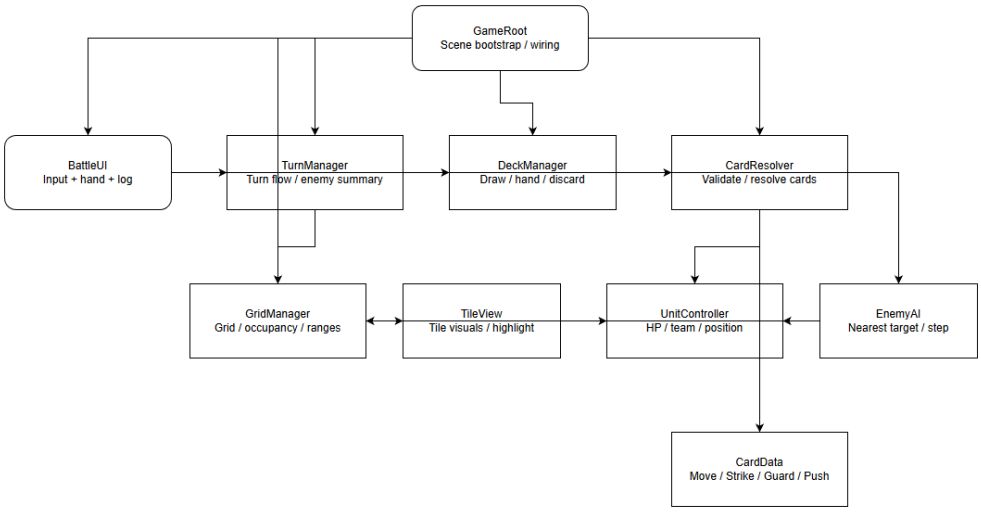

Figure 4.1: High-level runtime architecture for Tactical Cards.

This architecture diagram shows the major runtime modules and the direction of interaction between flow, board, cards, units, and UI. It communicates system responsibilities and runtime coordination at a level that a class-level inheritance diagram would not.

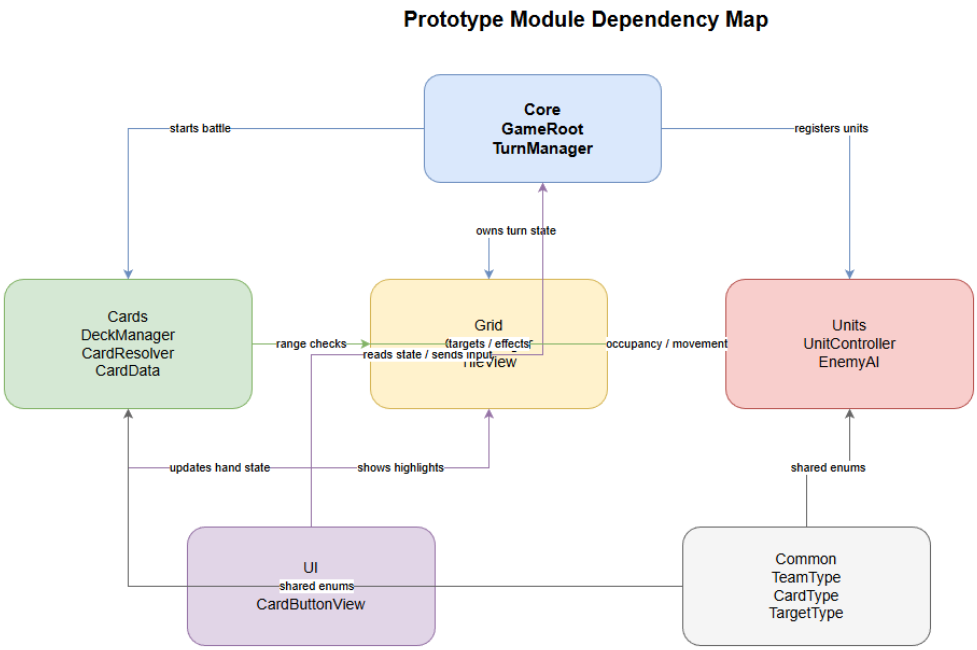

Figure 4.2: Simplified module dependency and responsibility map.

The module map complements the architecture view. The architecture diagram explains *what* subsystems exist and how they interact; the module map explains *where* those responsibilities live in the codebase. Table 4.1 extends those two figures with the final component inventory.

Area	Key classes	Responsibility	Origin
Core flow	‘GameRoot‘, ‘GameFlowController‘, ‘GameFlowState‘, ‘TurnManager‘	Scene bootstrap, title-battle-result flow, battle session start, turn state, and win/loss resolution	Group-authored
Cards	‘CardData‘, ‘DeckManager‘, ‘CardResolver‘	Card definitions, deck lifecycle, draw/discard handling, and explicit effect resolution	Group-authored
Grid	‘GridManager‘, ‘TileView‘	Board generation, occupancy, movement, range queries, and highlight state	Group-authored
Units	‘UnitController‘, ‘EnemyAI‘	Shared unit state, damage/heal/guard behaviour, and deterministic enemy targeting and movement	Group-authored
Runtime HUD	‘BattleUI‘, ‘BattleHudSnapshot‘, ‘CardButtonView‘, ‘UnitHealthBarView‘	Battle-state presentation, runtime card views, prompts, disabled-card feedback, and world-space health bars	Group-authored
Presentation	‘UnitVisualShell‘, ‘WorldThemeResources‘, ‘UiThemeResources‘	Board dressing, colour palette, unit silhouettes, and themed UI resource binding	Local presentation over open-licensed UI assets

Table 4.1: Main runtime components used by the delivered system.

The most important architectural boundary is the separation between battle rules and presentation. ‘TurnManager‘ owns turn progression, card-play budget, enemy phase execution, and battle-end state. ‘DeckManager‘ owns the draw pile, hand, and discard pile. ‘CardResolver‘ resolves card effects explicitly through readable branches instead of a large effect hierarchy. ‘GridManager‘ and ‘TileView‘ maintain board occupancy and targeting support. Together these classes form the deterministic gameplay core.

4.2 Implementation Strategy and Iteration Process

The project was developed through short, evidence-driven iterations rather than as a strict once-through sequence. A design baseline existed early, but the team repeatedly returned to requirements, UI layout, and test scope whenever a playable slice exposed a visible weakness. This working style aligns more closely with agile principles of frequent working software and responsiveness to change than with a rigid waterfall handoff model [3, 5].

The repository history makes those iterations visible. Instead of one large implementation drop, the worktree shows a sequence of focused increments: repository and baseline setup, prototype stabilisation, validation freeze, front-end flow completion, visual theming, and final HUD repair. Table 4.2 summarises the most representative slices.

Window	**Representative commits**	**Focus**	**Outcome**
01/04–02/04	`22b36f4`, `b045777`, `814c972`, `f2ac529`	Repository curation, clean baseline, battle-end correction, first regression protection	A stable prototype base with an early correctness fix
02/04–03/04	`968d396`, `7438df9`, `2d6a444`, `e407e63`	Architecture alignment, screenshots, diagrams, and prototype evidence	A reportable prototype with aligned technical artifacts
17/04	`c78434f`	Validation freeze and archived rerun evidence	A named baseline for execution and coverage evidence
23/04	`b8c1ab3`	Session flow shell, gameplay stability, and stronger automated coverage	A complete title-to-result session loop
23/04–24/04	`c4fa026`, `572acb2`	UI theming, board dressing, and unit presentation	A clearer visual identity for the tactical slice
24/04	`98753ae`	HUD readability closure and layout repair	The most visible UX defects were closed
24/04	`df9ea5a`	Final report package preparation	The final-report source tree and appendix scaffolding were assembled

Table 4.2: Representative iterations visible in the repository history.

This iterative process mattered because the product was not improved mainly by widening scope. Instead, each iteration targeted one of three questions:

- what must be built next to make the current battle slice more complete;
- what defect currently blocks understanding, stability, or submission credibility;
- what evidence is needed to prove that the change actually holds.

That is why flow, HUD readability, and validation work kept rising above more attractive content additions. The backlog was reprioritised by playable evidence, not only by original intention.

4.3 Prototype and UI Structure

The following figures illustrate the information structure and the runtime composition of the playable scene.

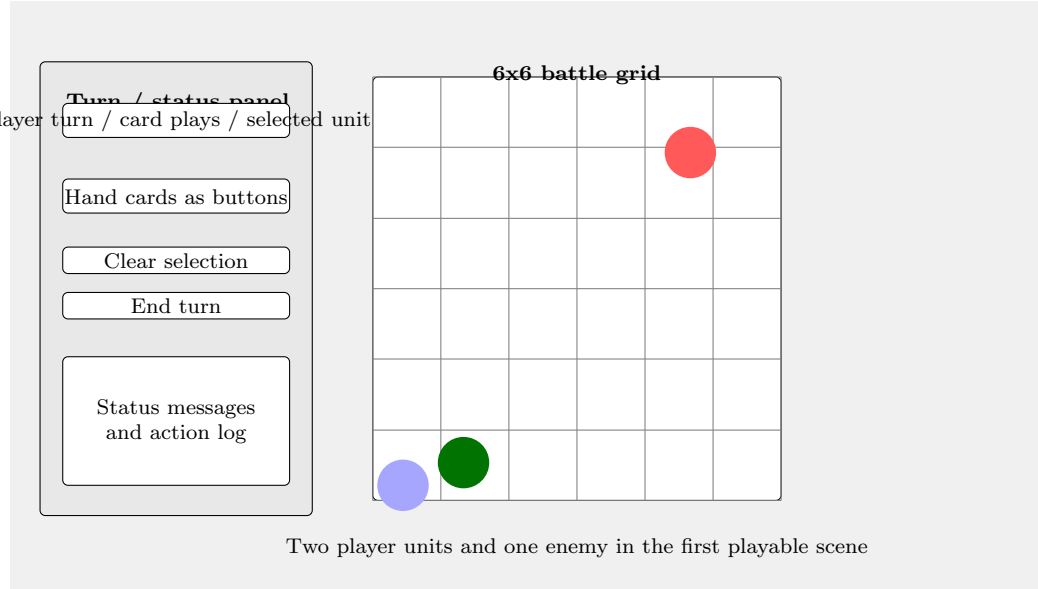

Figure 4.3: Low-fidelity wireframe showing the intended information zones.

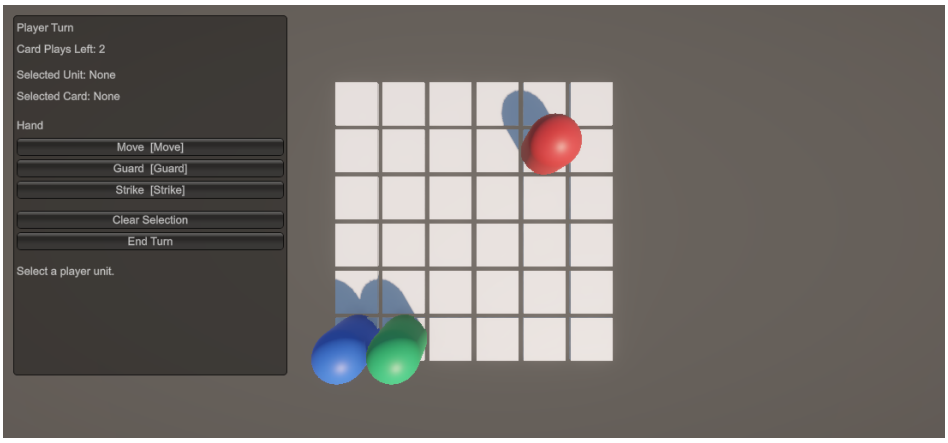

Figure 4.4: Battle scene with UI, hand, and 6x6 board.

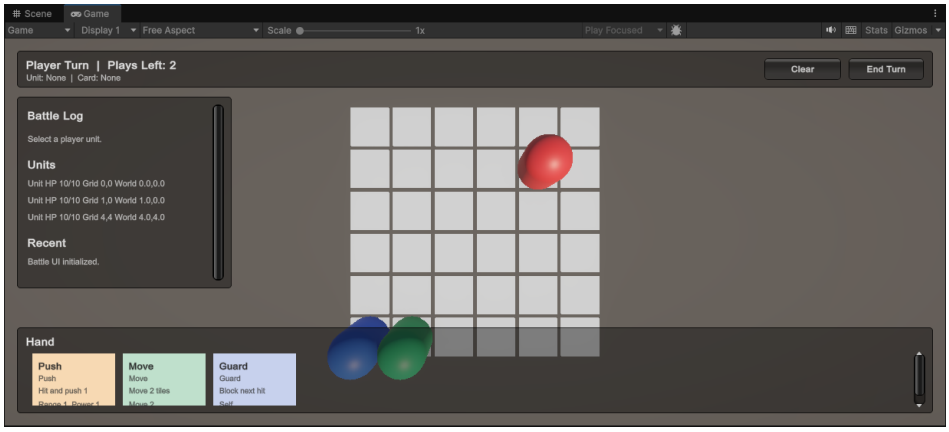

Figure 4.5: Runtime card hand and top-bar state.

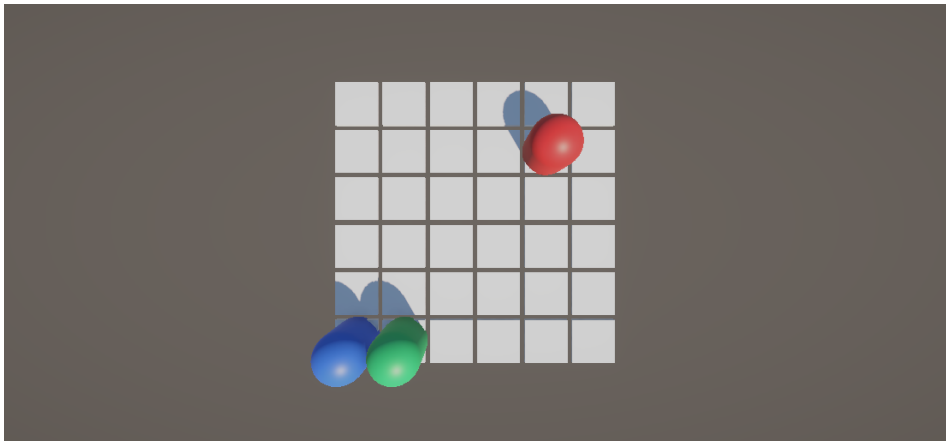

Figure 4.6: Board-only view of the playable battle scene.

The wireframe shows the intended information architecture: prompt information, a central board, and a persistent hand zone. The runtime screenshots show how that structure appears in the delivered build.

4.4 Reserved Updated Screenshot Set

The figures above preserve the earlier runtime evidence already referenced by the report. To support a clearer final comparison, the following refreshed captures from the latest build sit alongside the earlier figures rather than silently replacing them.

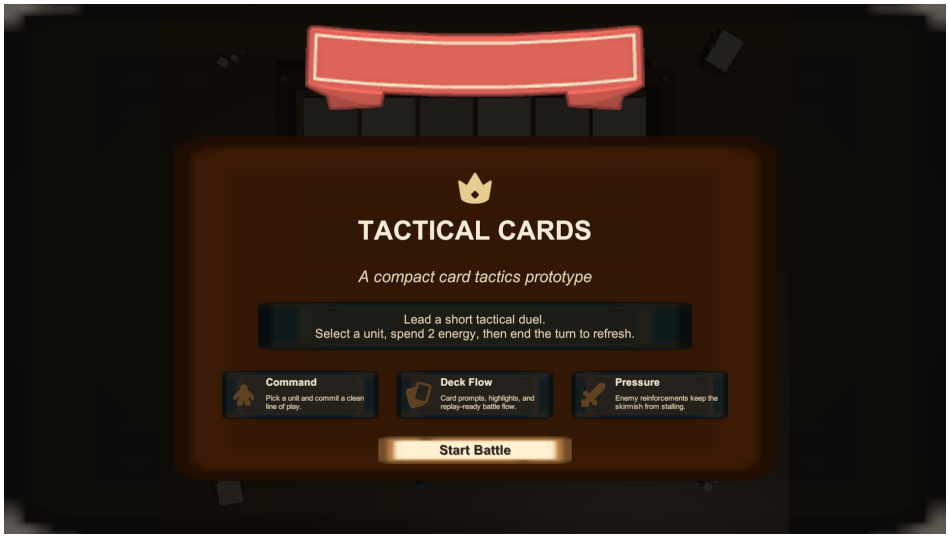

Figure 4.7: Updated main-menu screen from the latest build, shown alongside the earlier information architecture in Figure 4.3.

Figure 4.8: Updated battle overview from the latest build, shown alongside the earlier runtime battle screenshot in Figure 4.4.

Figure 4.9: Updated card-interface capture from the latest build, shown alongside the earlier hand-and-top-bar view in Figure 4.5.

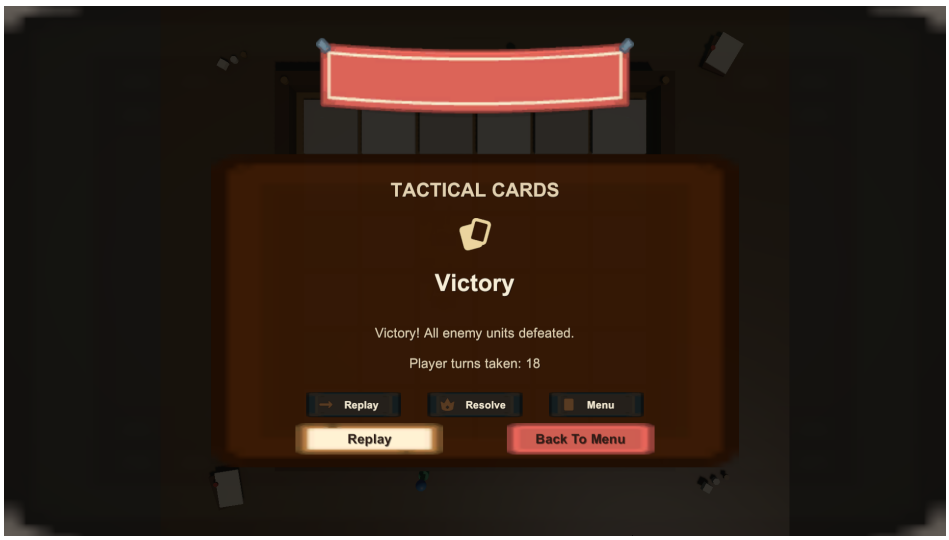

Figure 4.10: Victory result screen from the latest build, documenting the completed positive end-state flow.

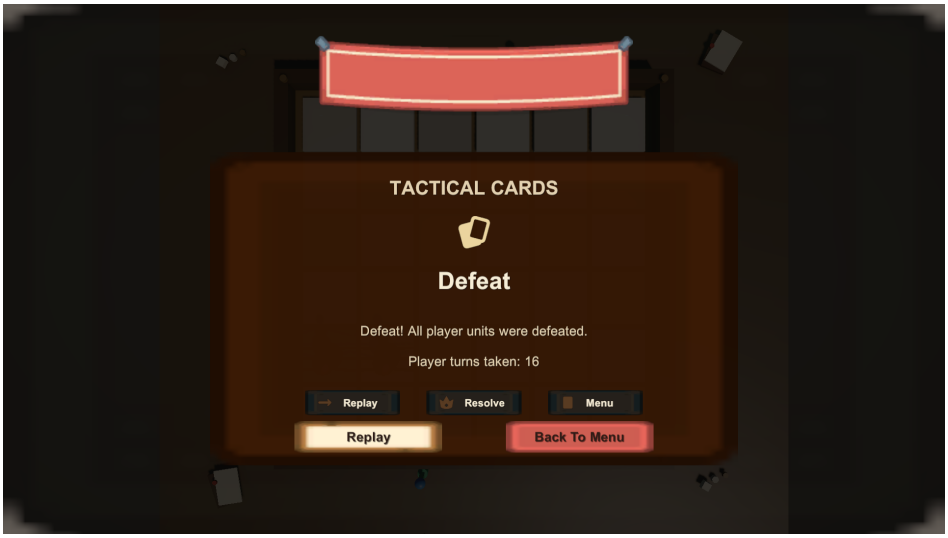

Figure 4.11: Defeat result screen from the latest build, documenting the alternate end-state flow.

4.5 Code Structure

The project is built around a small set of scene-bound ‘MonoBehaviour‘ scripts plus a few light model or helper types, so folder and responsibility views communicate the runtime better than a large inheritance tree would.

```
Assets/Scripts
|-- Cards
|   |-- CardData.cs
|   |-- CardResolver.cs
|   ‘-- DeckManager.cs
|-- Common
|   ‘-- GameEnums.cs
|-- Core
|   |-- GameFlowController.cs
|   |-- GameFlowState.cs
|   |-- GameRoot.cs
|   ‘-- TurnManager.cs
|-- Grid
|   |-- GridManager.cs
|   ‘-- TileView.cs
|-- Presentation
|   |-- UnitVisualShell.cs
|   ‘-- WorldThemeResources.cs
|-- UI
|   |-- BattleHudModels.cs
|   |-- BattleUI.cs
|   |-- CardButtonView.cs
|   |-- UiThemeResources.cs
|   ‘-- UnitHealthBarView.cs
‘-- Units
```

```
|-- EnemyAI.cs
`-- UnitController.cs
```

The implementation stays focused on a single battle slice. The battlefield, deck flow, and enemy response remain intentionally compact so that interaction clarity and system stability take priority over content breadth.

4.6 Unity Engine-Level Implementation Details

Concern	Actual implementation in the final build	Reason this choice was kept
Runtime UI root	‘GameFlowController‘ creates a ‘ScreenSpaceOverlay‘ ‘Canvas‘ with ‘CanvasScaler‘, ‘GraphicRaycaster‘, and ‘InputSystemUIInput-Module‘.	Keeps title, battle, and result flow in one scene and simplifies testing of flow state.
Board interaction path	‘BattleUI‘ reads ‘Mouse.current‘, checks ‘EventSys-tem.current.IsPointerOverGameObject()‘ and then uses ‘Physics.Raycast‘ for board clicks.	Directly links card selection to tile and unit selection without a middle layer or custom input layer.
Tile interaction surface	‘GridManager‘ creates tiles as ‘PrimitiveType.Cube‘, which provides colliders for click and occupancy interaction.	Simple primitive generation makes the board easy to build, raycast, and validate.
Presentation props	World dressing uses generated primitives with colliders removed or disabled through the presentation helper path.	Prevents decorative props from blocking tactical interaction while preserving scene dressing.
UI-state separation	‘BattleUI‘ builds ‘BattleHudSnapshot‘ and card-view models instead of reading scattered fields ad hoc.	Improves testability and reduces the risk of late HUD regressions.
Debug entry path	Direct-battle debug mode is routed through the shared flow bootstrap instead of bypassing the real HUD.	Ensures smoke and automated checks exercise the same runtime tree the player actually sees.

Table 4.3: Unity engine-level implementation details.

4.7 Simulation Design

The delivered game simulates one compact tactical encounter on a 6x6 grid. Two player-controlled units occupy the board at battle start and fight one or

more enemies under a turn-based loop. The simulation is deliberately deterministic: there are no random enemy tactics, no procedural map changes, and no branching objectives. This makes the battle state easier to understand for players and easier to validate for the development team.

At the start of each player turn, ‘TurnManager‘ refreshes the action budget and asks ‘DeckManager‘ to discard the previous hand and draw a new one. In the delivered build, the player draws three cards per turn and may spend two actions (‘maxCardPlaysPerTurn = 2‘). This keeps the turn short enough to remain readable while still allowing combinations such as move plus attack or defend plus reposition.

The fallback starter deck contains nine runtime cards:

- three ‘Move‘ cards;
- one ‘Dash‘;
- one ‘Strike‘;
- one ‘Long Strike‘;
- one ‘Guard‘;
- one ‘Push‘;
- one ‘Recover‘.

‘CardResolver‘ keeps the effect logic explicit. Movement and dash actions resolve against tiles, strike and push actions resolve against enemy units, and guard or recover resolve against the acting unit. Enemy behaviour remains deterministic: ‘EnemyAI‘ selects the nearest alive player unit by Manhattan distance and either attacks when adjacent or steps toward the target.

4.8 UI Design

The front-end flow is assembled at runtime by ‘GameFlowController‘. It creates the runtime canvas, builds title, battle, and result panels, and keeps them inside the same main scene. The title panel includes the game brand, a short summary, a feature strip, and a clear call to action. The result panel exposes the battle outcome plus replay and menu actions.

During battle, ‘BattleUI‘ builds a three-zone runtime HUD:

- a top bar for turn state, remaining energy, selection state, prompts, and action buttons;
- a left panel for unit summary, status, and the battle log;
- a bottom panel for the active hand of cards.

The runtime HUD does not pull ad hoc strings directly from scattered private state. Instead, it builds a ‘BattleHudSnapshot‘ containing turn labels, prompt text, selected-unit information, card states, and unit summaries. ‘CardButtonView‘ uses this snapshot to render card title, description, use hints, and disabled reasons directly in the hand. ‘UnitHealthBarView‘ projects health state back into world space so HP can be read from the board rather than only from the log.

Visual theming is added without replacing the project's own runtime structure. The interface uses themed UI resources for banners, panels, buttons, and symbols, while the board, props, and unit silhouettes remain driven by the project's own presentation code. The result is still a compact demo, but it now looks intentional rather than merely assembled.

Chapter 5

Test & System Evaluation

The validation approach does not rely on a single type of evidence. Instead, it combines direct EditMode tests for stable logic, PlayMode smoke for whole-scene readiness, and manual smoke for visual composition and overlay-heavy UI behaviour.

5.1 Test Strategy

The core strategy is *TDD-first where behaviour is deterministic, smoke-first where the result is primarily visual.* Turn flow, deck lifecycle, movement constraints, damage handling, and battle-end state all have explicit success conditions and can be tested directly. By contrast, title-panel readability, overlay composition, and the visual relationship between the board and the hand dock cannot be reduced safely to screenshot-perfect automation in the current project setup.

Validation layer	**Question being answered**	**Main techniques**	**Evidence form**
Verification	Does the implemented code behave according to the intended rules?	EditMode unit tests, focused component tests, reruns after bug fixes	Pass/fail results, suite inventory, module coverage matrix
Validation	Does the delivered battle flow make sense in use?	PlayMode smoke, scenario walkthrough, manual UX smoke	Smoke records and runtime confirmation

Table 5.1: Validation framing used by the current system.

Layer	Main purpose	Representative evidence	Used for
EditMode tests	Directly verify deterministic rules and state transitions	‘TurnManagerTests‘, ‘DeckManagerTests‘, ‘CardResolverTests‘, ‘GridManagerTests‘, ‘GameFlowControllerTests‘, ‘BattleUITests‘	Core logic, flow, and UI regressions
PlayMode smoke	Verify that the scene reaches a playable runtime state	‘BattleFlowPlayModeTests‘	Scene bootstrap and battle readiness
Manual smoke	Confirm readability, layout, and aesthetic stability	HUD and ambience smoke records	Overlay composition and final UX judgement

Table 5.2: Validation layers used by the current build.

5.2 Current Rerun State

The automated suite is organised by runtime responsibility. This mirrors the source-tree structure and makes failures easier to interpret. The current validation set reports:

- ‘81/81‘ passing EditMode tests;
- ‘1/1‘ passing PlayMode smoke test.

Checkpoint	EditMode	PlayMode	Primary focus	Evidence source
Archived validation freeze	‘57/57‘	‘1/1‘	Battle-loop validation and baseline requirements coverage	Archived validation summary
Presentation pass	‘75/75‘	‘1/1‘	World-layer presentation and scene dressing	Ambience smoke record
HUD feedback pass	‘81/81‘	‘1/1‘	HUD readability, disabled-card feedback, and layout regression closure	HUD feedback smoke record

Table 5.3: Current rerun state of the validation set.

The suite now covers the main runtime concerns directly:

- battle start, battle end, and turn-loop correctness;
- draw, discard, reshuffle, and hand lifecycle;
- movement, attack, push, guard, heal, and occupancy rules;
- enemy target selection and movement decisions;
- battle HUD prompts, disabled-card reasoning, and runtime layout guards;
- title-flow and result-flow state transitions.

5.3 Issue Discovery and Closure Loop

The most convincing part of the final validation package is not a single score or count. It is the way concrete problems were discovered, translated into engineering work, and then closed with evidence. The project repeatedly followed the same loop:

1. a visible problem was found through playtesting or smoke;
2. the deterministic part of that problem was isolated;
3. a regression test or structural assertion was added where possible;
4. runtime code was fixed;
5. the fix was checked again through rerun or smoke.

Issue	How it was found	Root cause	Fix	Closure
Battle-end continuation	Targeted prototype regression run	Battle-end state was not being finalised robustly enough	Stabilised turn-end resolution and added regression coverage	Prototype accuracy EditMode run
Title overlay collision	Manual smoke	Summary, feature strip, and start action competed for the same vertical space	Split the title overlay into separate content regions	Hierarchy checks plus smoke
Top-bar text overlap	Manual smoke	Prompt and state text were anchored into overlapping rows	Restacked the top bar and reserved button width explicitly	‘BattleUITests‘ plus smoke
Hand dock obscuring units	Manual smoke	The themed dock and cards were too large for the camera framing	Reduced dock and card dimensions and added layout guards	Layout test plus smoke
Action availability hidden in log	Scenario walkthrough and smoke	The prototype exposed state only indirectly through log text	Added energy, prompt detail, disabled-card overlays, and health bars	UI tests plus smoke
Debug flow bypassing real HUD	Runtime comparison during direct battle testing	Debug entry skipped the same UI tree used in normal play	Routed debug start through the shared flow controller	Flow tests and runtime check

Table 5.4: Representative issue-discovery and closure loop in the delivered project.

This loop is important because it shows that visual and UX problems were not treated as vague polish notes. They were turned into traceable engineering tasks with explicit closure criteria.

5.4 Code Coverage

The project has an archived code-coverage baseline for the current battle slice. That archive recorded ‘11‘ runtime classes in the report, ‘74.2%‘ line coverage, and ‘92.4%‘ method coverage.

Coverage metric	Value	Evidence meaning
Runtime classes in report	‘11‘	The archived coverage report spans the runtime classes cited in this chapter.
Covered lines	‘868‘	Taken from the archived multi-report coverage summary.
Coverable lines	‘1169‘	Used as the denominator for the line-coverage claim.
Line coverage	‘74.2%‘	The project has archived quantitative evidence rather than package-only tooling claims.
Covered methods	‘146‘	Shows that the suite closes most method-level behaviour in the battle slice.
Total methods	‘158‘	Method totals come directly from the archived coverage summary.
Method coverage	‘92.4%‘	Method coverage is materially stronger than the line aggregate, which is expected for UI-heavy scripts.

Table 5.5: Archived coverage totals for the current battle slice.

Figure 5.1: Class-level line coverage from the archived report. ‘BattleUI‘ remains the lowest-covered runtime class and therefore stays a watch point.

The archived coverage figure has not yet been regenerated after the latest UI and flow additions. For that reason, the ‘74.2%‘ line-coverage figure should be treated as the current archived baseline rather than as a freshly regenerated post-polish number.

5.5 Evaluation and Technical Self-Critique

The delivered build satisfies the main goal of the project: it provides a complete and readable tactical battle loop rather than only a technical prototype. The player can begin from a title screen, enter battle, select units and cards, understand the turn state through the HUD, observe enemy response, and reach a result screen with replay or menu options.

Area	Outcome	Reasoning
Playable front-end flow	Satisfied	Title, battle, and result states exist inside the main scene and are covered by flow and smoke evidence.
Battle rules and turn loop	Satisfied	Movement, attack, guard, push, recover, draw/discard, enemy turns, and battle-end handling are directly tested.
HUD readability and action feedback	Satisfied	Energy, prompts, disabled-card reasons, and world-space health bars close the main readability gaps.
Visual and thematic cohesion	Satisfied	Themed UI assets, board ambience, and silhouette-based units give the slice a coherent visual direction.
Residual limitations	Open but non-blocking	The project remains a single-battle demo, overlay-heavy screenshot capture still needs manual smoke, and coverage needs a fresh export.

Table 5.6: Evaluation summary for the delivered build.

The strongest technical lesson from the final testing cycle is that the suite is substantial, but not perfect. The weaker areas are:

- **Mock and scene fidelity.** Several tests are scene-coupled component tests rather than isolated pure unit tests.
- **Assertion depth.** UI-facing tests prove structure and state transitions, but they cannot fully prove visual readability or overlay composition.
- **Coverage concentration.** ‘BattleUI‘ remains the lowest-covered runtime class in the archived report.
- **PlayMode breadth.** PlayMode coverage is intentionally narrow and proves readiness more than full end-to-end variety.

The project is therefore well validated for a compact vertical slice, but the testing package is stronger on deterministic logic than on fully automated presentation proof.

Chapter 6

Extensibility & Maintenance

Although Tactical Cards is a compact prototype, the delivered system is structured so that it can be maintained and extended without structural rework.

6.1 Delivered Maintainability Measures

The codebase is structured around functional responsibility rather than scene-specific scripts. Core bootstrap and flow classes are separated from battle rules, card resolution, unit logic, grid state, UI presentation, and purely visual wrappers. This makes it possible to modify one layer without rewriting the others. For example, the title flow, HUD layout, disabled-card messaging, and health-bar presentation can change without replacing the underlying turn model or card resolver.

Assembly definitions provide a second maintainability guardrail. Runtime code is grouped in the main ‘TacticalCards‘ assembly, while EditMode and PlayMode test suites are isolated in separate test assemblies. This separation reduces build pollution, avoids test-only dependencies leaking into player-facing code, and makes the validation boundary easier to explain.

Third-party assets are also contained deliberately. Imported interface art is kept separate from the battle rules and runtime state logic. This means the visual theme can be updated or replaced without having to disentangle it from gameplay behaviour.

6.2 Deferred Work and Future Evolution

The most important deferred work is content expansion, not systems rescue. The delivered game does not include a campaign structure, multiple maps, progression systems, deep deck customisation, or a large enemy catalogue. Those omissions are intentional because each of them would multiply test surface, UI complexity, and content production cost without improving the quality of the core tactical slice.

Follow-up direction	What would be added	Rough effort	Why it was deferred
Encounter variety	More objectives, alternate spawn patterns, and one or two additional maps.	1–2 weeks	Would widen the test surface and increase tuning cost.
Character presentation	Replace procedural silhouettes with coherent character assets and simple animation.	1 week	HUD readability and battle clarity had higher priority.
Coverage refresh	Regenerate the coverage archive after the latest UI and flow additions.	1–2 days	The existing archive is still the last stable quantitative baseline.
Broader PlayMode journeys	Add more end-to-end PlayMode paths for win, loss, replay, and menu return.	2–3 days	EditMode coverage and targeted smoke had more immediate value.

Table 6.1: Concrete extension paths with rough effort estimates.

The current architecture already shows that the project can absorb flow, HUD, and theming extensions in thin slices without destabilising the entire battle loop.

Chapter 7

Problem & Reflections

Many of the hardest problems in this project were not purely visual and not purely logical. In practice, presentation, interaction, testing, and planning were tightly coupled. Once the interface became more elaborate, every UI change exposed another layer of engineering work: raycast boundaries, runtime layout assumptions, battle-state messaging, or proof that a fix had actually held. This chapter reflects on those issues at technical, project-management, and team-management levels.

7.1 Technical Problem

One recurring technical problem was that visual polish changed interaction behaviour. As the title overlay, themed HUD, and hand dock became more elaborate, previously hidden assumptions in the prototype started to fail. Concrete examples included title overlay content colliding vertically, the top-left prompt stack overlapping turn and energy labels, the hand dock obscuring lower units, and action availability being visible only in the battle log.

The important lesson was that these defects could not be handled with visual tweaking alone. Several bugs existed at the boundary between layout and logic. A panel could look harmless but still intercept clicks; a debug shortcut could speed up testing but bypass the same flow path used by the real HUD; a screenshot could show the board correctly while missing overlay elements entirely.

Another technical limitation came from tooling. Overlay screenshots were not always trustworthy for ‘Screen Space Overlay‘ verification in this setup. The practical response was to split validation responsibility: hierarchy assertions and runtime tests for structural proof, and manual smoke for final overlay readability.

7.2 Project Management

The main project-management difficulty was scope control. The original idea could easily have expanded into a larger tactics game with more cards, more enemy types, more maps, and some form of progression system. In practice, ev-

ery time the team widened the presentation or interaction layer, it also created more testing and debugging work. That made it necessary to keep returning to the same rule: finish one battle slice properly before adding more content.

This affected planning throughout the project. Some work that initially looked like polish, such as title flow, result flow, disabled-card feedback, or health-bar visibility, later turned out to be necessary for a complete and readable submission. As a result, backlog decisions were not based only on what sounded interesting to add next. They were based on which issue most directly blocked the game from being understandable, stable, or demonstrably complete.

Testing and evidence collection also became planning tools. When a feature had no clear validation path, that was treated as a warning sign that the implementation was still too vague. This is one reason the project stopped short of adding wider content late in development. It was more valuable to close visible usability defects with proof than to keep extending the ruleset without the same level of control.

7.3 Team Management

Our team coordination was not great at the beginning of the project. For the first few weeks, most of the communication happened through WeChat group messages, and files were shared as loose Word documents with a naming convention we agreed on in Week 1 (see Appendix B.3). At that point we did have a GitHub repository set up, but in practice nobody was really committing regularly. Hongshuo and Cheng were doing most of the coding work locally, and Guangjun was writing requirements documents in separate '.docx' files. Zehan was working on testing but did not have much to test until the prototype was more stable. So the actual project state lived in people's local machines and chat history rather than in one traceable place.

This caused confusion around the middle of the project. After the second weekly submission, we made a more serious effort to move everything onto GitHub with proper branches and pull requests, and also started collecting screenshots, test records, and meeting notes into the documentation tree in the repository. Around the same time, we moved our report writing from Word documents to Overleaf so that everyone could edit the same LaTeX source simultaneously. Before that switch, the report drafting process was messy. One person would write a section in Word, send it to the group chat, and then another person would copy-paste it into whichever version they had open. Merging those scattered drafts often took longer than the writing itself. With Overleaf we could at least see each other's edits in real time, which reduced duplicated paragraphs and version mismatches. It also forced us to learn LaTeX along the way, which slowed things down at first but made the final formatting much more consistent than the earlier Word submissions.

That said, the switch to a proper workflow for both code and documentation came later than it should have. Because of this, some of our early meeting records and planning decisions are only preserved as recovered notes rather than as proper version-controlled documents, as described in Appendix B.1. If

we were to start over, we would enforce a repository-first workflow from Week 1 and set up Overleaf at the same time instead of halfway through. The other thing we would change is having clearer code ownership earlier. For most of the project, it was not always obvious who was responsible for which scripts, and this sometimes led to duplicated effort or parts of the code that nobody felt confident enough to modify.

Chapter 8

Conclusion

This project delivers a complete tactical card battle prototype in Unity. The final build provides a coherent front-end flow, a functioning card-driven skirmish on a grid, readable battle feedback, a result state, and a system structure that separates flow, battle rules, UI, and presentation cleanly.

The project also satisfies the wider expectations of a final software-engineering submission. The report now presents a complete software artifact, an updated system and UI design, an explicit explanation of how requirements evolved from the initial concept into the delivered implementation, an iterative development strategy grounded in repository history, a validation package that combines automated reruns with smoke-backed issue closure, and a maintainability outlook supported by clear module boundaries and appendices.

The design remains intentionally small, but that small scope is one of its strengths: it allows the game to finish as a real vertical slice rather than remaining a larger but incomplete concept. Overall, Tactical Cards demonstrates a compact but complete tactical design, a stable implementation path, and a structure that can support future extension without replacing the core battle loop. The accompanying appendices strengthen that claim by preserving the user manual, meeting evidence, test appendix, planning summary, version-control summary, and supporting-document register needed to keep the final package self-contained [7].

Reference

[1] Paul J. C. Adachi and Teena Willoughby. More than just fun and games: The longitudinal relationships between strategic video games, self-reported problem solving skills, and academic grades. *Journal of Youth and Adolescence*, 42(7):1041–1052, 2013.

[2] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, Boston, 2003.

[3] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for agile software development. https://agilemanifesto.org/, 2001. Accessed 2026-04-26.

[4] Isabela Granic, Adam Lobel, and Rutger C. M. E. Engels. The benefits of playing video games. *American Psychologist*, 69(1):66–78, 2014.

[5] Winston W. Royce. Managing the development of large software systems. In *Proceedings of IEEE WESCON*, pages 1–9, 1970.

[6] Richard M. Ryan, C. Scott Rigby, and Andrew Przybylski. The motivational pull of video games: A self-determination theory approach. *Motivation and Emotion*, 30(4):344–360, 2006.

[7] University of Nottingham. Comp4135 coursework 1 specification, 2026. Local module specification stored with the course materials.

Appendix A

User Manual

A.1 Purpose

This manual explains how to run and play the current Tactical Cards demo in the Unity Editor. The delivered coursework build is a compact single-battle prototype rather than a full campaign game, so the manual focuses on the playable battle loop and the runtime HUD.

A.2 Opening the Project

1. Open the Unity project located in `coursework/`.
2. Load `Assets/Scenes/SampleScene.unity`.
3. Press the Unity Editor `Play` button.

When the scene starts correctly, the player first sees the title screen rather than entering combat immediately.

A.3 Starting a Battle

1. Wait for the title screen to finish loading.
2. Press `Start Battle`.
3. The game transitions into the battle state and the battle HUD becomes active.

If the battle has already finished, the result screen allows the player to replay the battle or return to the title screen.

A.4 Core Interaction Flow

The intended interaction order during the player turn is:

1. select a player unit on the board;
2. select a card from the hand;
3. select a valid target tile or enemy unit if the card requires one;

4. observe the result and continue until the turn ends.

The player can press `Clear` to reset the current selection and `End Turn` to pass control to the enemy phase.

A.5 Reading the Battle HUD

A.5.1 Top Bar

The top bar provides the most important turn-state information:

- current turn state;
- remaining action budget, shown as `Energy`;
- current selection state;
- prompt text describing what the player should do next.

A.5.2 Left Panel

The left panel provides supporting battle information:

- unit summary;
- battle log;
- recent action history.

A.5.3 Bottom Hand Panel

The bottom panel displays the current hand of cards. Each card includes:

- card name;
- action type;
- effect summary such as range, damage, movement, or recovery value.

When a card cannot be used in the current state, the card shows disabled feedback directly instead of relying only on the battle log.

A.5.4 World-space Unit Feedback

Each unit displays a world-space health bar above the model. This allows the player to read board state without scanning the text log after every action.

A.6 Card Reference

The current fallback starter deck supports the following actions:

- **Move**: move the selected unit up to 2 tiles.
- **Dash**: move the selected unit up to 3 tiles.
- **Strike**: deal 2 damage to an adjacent enemy.
- **Long Strike**: deal 2 damage to an enemy within 2 tiles.
- **Guard**: reduce the next incoming hit against the selected unit.

- **Push**: deal 1 damage and push the enemy backward if possible.
- **Recover**: restore 2 health to the selected unit.

A.7 Understanding the Turn Loop

1. At the start of the player turn, the game refreshes the action budget and hand state.
2. The player performs a limited number of card-driven actions.
3. The player ends the turn.
4. Enemy units execute their actions automatically.
5. The game checks whether the battle should continue or end in victory or defeat.

Some battles may introduce reinforcement pressure during play. This is an intended part of the final tactical slice.

A.8 Ending the Battle

When the battle reaches a win or loss condition, the runtime flow changes to the result screen. The result screen provides:

- a visible battle outcome;
- replay/start-over behaviour;
- a return path to the title screen.

A.9 Troubleshooting

- If the title screen does not appear, verify that `SampleScene.unity` is the loaded scene and that the project compiled successfully.
- If a card cannot be played, check the current prompt text, selection state, and the card's disabled feedback.
- If board state becomes hard to follow, use the top-bar prompt plus the world-space health bars before relying on the battle log.

Appendix B

Meeting Minutes

B.1 Evidence Status

This appendix preserves the meeting evidence currently archived in the workspace. One early checkpoint is reconstructed from dated project artifacts rather than preserved as a full formal minute set, so it remains explicitly labelled as *recovered*. That distinction is kept visible instead of being hidden.

Record	Date	Type	Current use
Recovered early implementation checkpoint	15/03/2026	Recovered note	Explains the first playable scope and early implementation decisions.
Kickoff meeting	17/03/2026	Formal note	Shows initial direction-setting and role alignment.
Scope alignment meeting	18/03/2026	Formal note	Shows agreement on tactical-battle scope and documentation format.
Validation planning meeting	15/04/2026	Formal note	Shows testing strategy, defect triage, and final validation planning.

Table B.1: Meeting evidence preserved in the final-report appendix.

B.2 Recovered Early Implementation Checkpoint

Date: 15/03/2026
Type: recovered from local implementation artifacts
Attendance: exact attendance not preserved in the workspace

B.2.1 Purpose

Define the minimum playable slice that could demonstrate the project idea in software rather than only in prose.

B.2.2 Recovered Decisions

- Keep one battle scene for the first playable version.
- Fix the board at 6x6 so movement, targeting, and occupancy remain readable.
- Limit the first playable card set to Move, Strike, Guard, and Push.
- Use one shared ‘UnitController‘ for player and enemy units.
- Keep enemy behaviour deterministic.
- Use button-based card selection instead of drag-and-drop.

B.2.3 Recovered Follow-up

- Implement the first-pass script set and scene bootstrap.
- Refresh diagrams so they match the code actually built.
- Capture early prototype screenshots.
- Start separating validation notes from design notes.

B.3 Kickoff Meeting

Date: 17/03/2026
Attendance: Guangjun HU, Hongshuo HU, Zehan CHEN

B.3.1 Context

The team compared initial game ideas, narrowed the project direction, and assigned the first requirement and planning tasks.

B.3.2 Decisions

- Confirm Tactical Cards as the chosen project direction.
- Agree initial role allocation.
- Set the first delivery expectations for requirements and design material.

B.4 Scope Alignment Meeting

Date: 18/03/2026
Attendance: Guangjun HU, Cheng YE

B.4.1 Context

This meeting aligned the project around a tactical card battle demo and standardised a lightweight documentation format for later work.

B.4.2 Decisions

- Keep the software scope focused on a tactical card battle rather than a larger campaign game.
- Assign UML and design-diagram production.
- Use a repeatable meeting-note structure for later checkpoints.

B.5 Validation Planning Meeting

Date: 15/04/2026
Format: Online
Chair: Guangjun HU
Secretary: Zehan CHEN

B.5.1 Attendance

- Guangjun HU – present
- Hongshuo HU – present
- Cheng YE – present
- Zehan CHEN – present

B.5.2 Agenda

1. Review usability findings from the playable build.
2. Confirm automated-test scope and smoke responsibilities.
3. Allocate drafting tasks for the final validation material.
4. Set deadlines for open defects and reruns.

B.5.3 Key Decisions

- Treat deterministic behaviour as regression-test candidates and keep manual smoke for overlay-heavy UI concerns.
- Require rerun-backed evidence before a visible issue can be considered closed.
- Keep late work focused on flow, HUD, and readability rather than widening the feature set.

B.5.4 Action Items

Task	Owner	Due	Review owner
Draft report sections and supporting tables	Cheng YE	16/04/2026	Guangjun HU
Run EditMode tests and archive results	Hongshuo HU	15/04/2026	Guangjun HU
Update defect register and meeting notes	Zehan CHEN	16/04/2026	Cheng YE
Run final integration check in Unity	Hongshuo HU	17/04/2026	Whole team

Table B.2: Action items from the validation planning meeting.

Appendix C

Test Case

C.1 Validation Evidence Register

Evidence item	Date	Type	Use in the report
Prototype accuracy regression run	02/04/2026	EditMode run	Shows the battle-end correctness fix was rerun and closed.
Archived validation summary	17/04/2026	Execution record	Provides the archived baseline suite counts and coverage export.
Coverage export snapshot	17/04/2026	Coverage record	Provides the quantitative line and method coverage baseline.
Defect and retest register	17/04/2026	Defect log	Shows which inherited or current defects were closed through rerun evidence.
Visual ambience smoke	24/04/2026	Manual smoke	Shows the themed board and scene dressing stayed stable.
HUD feedback smoke	24/04/2026	Manual smoke	Shows the final HUD layout and interaction feedback remained readable.
Late-stage UI and HUD issue log	24/04/2026	Issue log	Explains how the final visual defects were found and closed.

Table C.1: Main validation records preserved for the final report.

C.2 Coverage Snapshot

The appendix preserves two complementary views of validation:

- an archived quantitative coverage snapshot;
- later rerun counts after the final flow and HUD passes.

Metric	Value	Interpretation
Archived runtime classes	‘11‘	The archived coverage report covers the core runtime slice.
Archived line coverage	‘74.2%‘	Useful quantitative baseline before the final polishing passes.
Archived method coverage	‘92.4%‘	Shows strong method-level closure for the deterministic battle systems.
Latest EditMode rerun	‘81/81‘	Shows the suite expanded after later flow and HUD work.
Latest PlayMode rerun	‘1/1‘	Shows the playable scene still boots and reaches battle readiness.

Table C.2: Coverage and rerun snapshot preserved in the appendix.

C.3 Representative Automated Test Inventory

Area	Representative suites	What they protect
Battle bootstrap	‘GameRootTests‘, ‘GameFlowControllerTests‘	Flow transitions, runtime UI creation, replay/menu behaviour
Turn loop	‘TurnManagerTests‘	Action budget, battle-end rules, enemy-phase handoff
Cards and deck	‘DeckManagerTests‘, ‘CardResolverTests‘	Draw/discard lifecycle and card-effect branches
Grid and tiles	‘GridManagerTests‘, ‘TileViewTests‘	Placement, occupancy, range, and highlight behaviour
Units and AI	‘UnitControllerTests‘, ‘EnemyAITests‘	Damage, healing, guard, target choice, and movement
HUD and card views	‘BattleUITests‘, ‘CardButtonViewTests‘	Prompt layout, disabled-card states, and layout regression guards
Playable-scene readiness	‘BattleFlowPlayModeTests‘	Scene bootstrap and battle entry in Play Mode

Table C.3: Representative automated test coverage in the delivered project.

C.4 Manual Smoke Checklist

The following player-visible checks were retained as manual smoke responsibilities:

- title screen loads and the start button is readable;
- battle HUD appears without text overlap;
- opening hand dock does not obscure the lower board excessively;
- victory and defeat both reach the result state cleanly;
- disabled cards explain why they cannot be played;
- world-space health bars remain readable during combat.

Appendix D

Development Planning

D.1 Iterative Delivery Model

The project was delivered in short increments rather than as a once-through sequential build. The repository history shows repeated cycles of implementation, playable checking, design revision, validation, and interface refinement.

Iteration window	Representative commits	Primary focus	Outcome
01/04–02/04	`22b36f4`, `b045777`, `814c972`, `f2ac529`	Repository curation, clean baseline, battle-end correction, initial regression protection	A stable prototype baseline with an early correctness fix
02/04–03/04	`968d396`, `7438df9`, `2d6a444`, `e407e63`	Architecture alignment, screenshots, diagrams, and code/report consistency	A reportable prototype with aligned visual and structural evidence
17/04	`c78434f`	Validation freeze and archived rerun evidence	A named baseline for execution and coverage evidence
23/04	`b8c1ab3`	Title-to-result flow shell, gameplay stability, and stronger test coverage	A complete session loop rather than battle-only entry
23/04–24/04	`c4fa026`, `572acb2`	Runtime UI theming, board dressing, and unit presentation	A stronger visual identity without changing the core rules
24/04	`98753ae`	HUD feedback closure, overlay repair, and layout fixes	The last major usability gaps were closed
24/04	`df9ea5a`	Final report package preparation	The report source tree and final-document scaffolding were assembled

Table D.1: Representative iterative slices visible in the repository history.

D.2 Backlog Reprioritisation Logic

The team did not treat the plan as fixed after the first playable build. The backlog was reprioritised when direct evidence showed a more urgent need. In practice, the reprioritisation rule was:

1. keep the core tactical slice playable at all times;
2. promote any defect that blocks understanding, battle completion, or report credibility;

3. defer attractive but expensive content work if it does not improve the current slice materially.

This logic is why session flow, HUD readability, and regression protection were prioritised above wider content expansion.

Appendix E

Version Control

E.1 Workflow Summary

Practice	How it was applied	Why it mattered
Canonical remote repository	The project was consolidated into one remote repository with visible branch and merge history.	Provided a dated engineering trail rather than isolated local files.
Branch-based change integration	Major report and code changes were grouped into identifiable commits and merge checkpoints.	Reduced ambiguity about what changed and when.
Review-first code changes	Later submission-facing changes were aligned to a branch-and-review workflow.	Improved accountability for risky integration work.
Evidence-tree preservation	Testing records, issue logs, screenshots, and report material were kept in the documentation tree.	Made the report easier to support with named artifacts.

Table E.1: Version-control and repository workflow used by the project.

E.2 Repository Graph Snapshot

The report does not rely on a hand-drawn branch diagram. Instead, the repository graph below is taken directly from the local Git history using a graph-formatted log command. This makes the version-control evidence traceable to the repository itself rather than to a reconstructed illustration.

```
* 8bea5e9 2026-04-26  (HEAD -> test)
* df9ea5a 2026-04-24  (origin/test)
* 98753ae 2026-04-24
* 572acb2 2026-04-24
* c4fa026 2026-04-23
```

```
* b8c1ab3 2026-04-23
* c78434f 2026-04-17
*   bd21bd8 2026-04-17  (origin/main, main)
|\
| * e4f21e5 2026-04-17
| * 85563ca 2026-04-03
| * 83bbb1c 2026-04-03
| * ffff57c 2026-04-03
| * 10f2816 2026-04-03
| * 7054891 2026-04-03
* |   7538945 2026-04-03
|\ \
| * | cddd0fa 2026-04-03  (origin/develop, develop)
| * | b91b578 2026-04-03
```

This snapshot is sufficient to demonstrate that the project used real branching and merge activity across multiple lines of work. If a cleaner hosting-platform screenshot is desired for presentation polish, it can replace or accompany this generated graph without changing the evidence claim.

E.3 Workflow Limits

The repository trail is stronger than the earliest coordination trail. Early planning relied more on informal coordination than on a fully preserved digital board history. The final report therefore distinguishes between:

- repository activity that is directly provable from commits and merges;
- planning or meeting context that is preserved only partially;
- recovered notes that remain labelled as recovered.

This appendix keeps that distinction visible instead of implying stronger historical evidence than the workspace can prove.

Appendix F

Docs

F.1 Supporting Document Register

Document group	Role	Use in the final package
Course specification and template	Assessment guidance	Used to align the report structure and submission scope.
User manual	Player-facing appendix	Explains how to run the project and understand the battle HUD.
Meeting register and recovered checkpoint notes	Process evidence	Show how scope and validation decisions were agreed and tracked.
Archived validation summaries	Testing evidence	Support execution counts, coverage claims, and regression discussion.
Smoke records and issue log	Late-stage closure evidence	Explain how visual and HUD issues were found and closed.
Asset notice and open-license register	Attribution evidence	Record third-party UI resources used by the project.
Repository notes and README	Project-operation evidence	Describe how the codebase is organised and how the project is run.

Table F.1: Supporting documentation preserved for the final submission package.